

Linked Lists

A linked list is a data structure where elements (nodes) are connected via explicit pointers (or references) rather than stored in contiguous memory.

Natural Explanation

Instead of storing all items in a row (like an array), each item has a pointer to the next item. This creates a chain: you navigate by following pointers from node to node.

Node Structure

Each node contains: - **Data**: the stored value - **Link(s)**: pointer(s) to other node(s)

`[data | link] -> [data | link] -> [data | link] -> null`

Singly Linked List

Each node has one outgoing pointer to the next node (or null if last).

Operations: - **Search:** traverse from head until found; worst case $O(n)$ - **Insert at front:** create new node, point to old head, update head pointer; $O(1)$ - **Insert after node x:** create new node, update x's pointer; $O(1)$ - **Delete from front:** update head to next node; $O(1)$ - **Delete node x:** need predecessor's pointer to update; worst case $O(n)$ (must find predecessor) - **Access position i :** traverse i steps; $O(i)$

Doubly Linked List

Each node has two pointers: one to the previous node, one to the next.

Advantages over singly linked: - Traverse backward - Delete a node if you have its pointer (no need to find predecessor); $O(1)$ - Some operations are symmetric and cleaner

Disadvantage: - Extra space for second pointer

Comparison with Arrays

Operation	Array	Singly Linked	Doubly Linked
Access by index	$O(1)$	$O(n)$	$O(n)$
Insert at front	$O(n)$	$O(1)$	$O(1)$
Insert/delete with pointer	$O(n)$	$O(1)$ (insert) / $O(n)$ (delete)	$O(1)$
Space	$O(n)$	$O(n)$	$O(n)$
Cache locality	Good	Poor	Poor

Use Cases

- When you need frequent insertions/deletions at known positions (not indices)
- When memory allocation must be flexible (no contiguous block needed)
- Implementing [\[\[Stacks\]\]](#) and [\[\[Queues\]\]](#) with $O(1)$ operations
- Graph adjacency lists (linked list of neighbors per vertex)

Related

- [\[\[Stacks\]\]](#)
- [\[\[Queues\]\]](#)
- [\[\[Dynamic Arrays\]\]](#)
- [\[\[Abstract Data Types and Concrete Data Types\]\]](#)